

Security Audit

Zetachain Standard Contracts (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	17
Conclusion	18
Our Methodology	19
Disclaimers	21
About Hashlock	22

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The ZetaChain team partnered with Hashlock to conduct a security audit of their Standard smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

ZetaChain is a decentralized blockchain network designed to enable omnichain interoperability, allowing seamless communication and interaction between multiple blockchains, including Ethereum, BSC, Polygon, and non-smart contract chains like Bitcoin. By providing built-in cross-chain messaging and token management capabilities, ZetaChain eliminates the need for external bridges, reducing complexity and potential security risks. The platform aims to empower developers to build omnichain decentralized applications (dApps) with native support for multi-chain functionality, ensuring a unified and secure user experience across diverse blockchain ecosystems.

Project Name: ZetaChain Standard Contracts

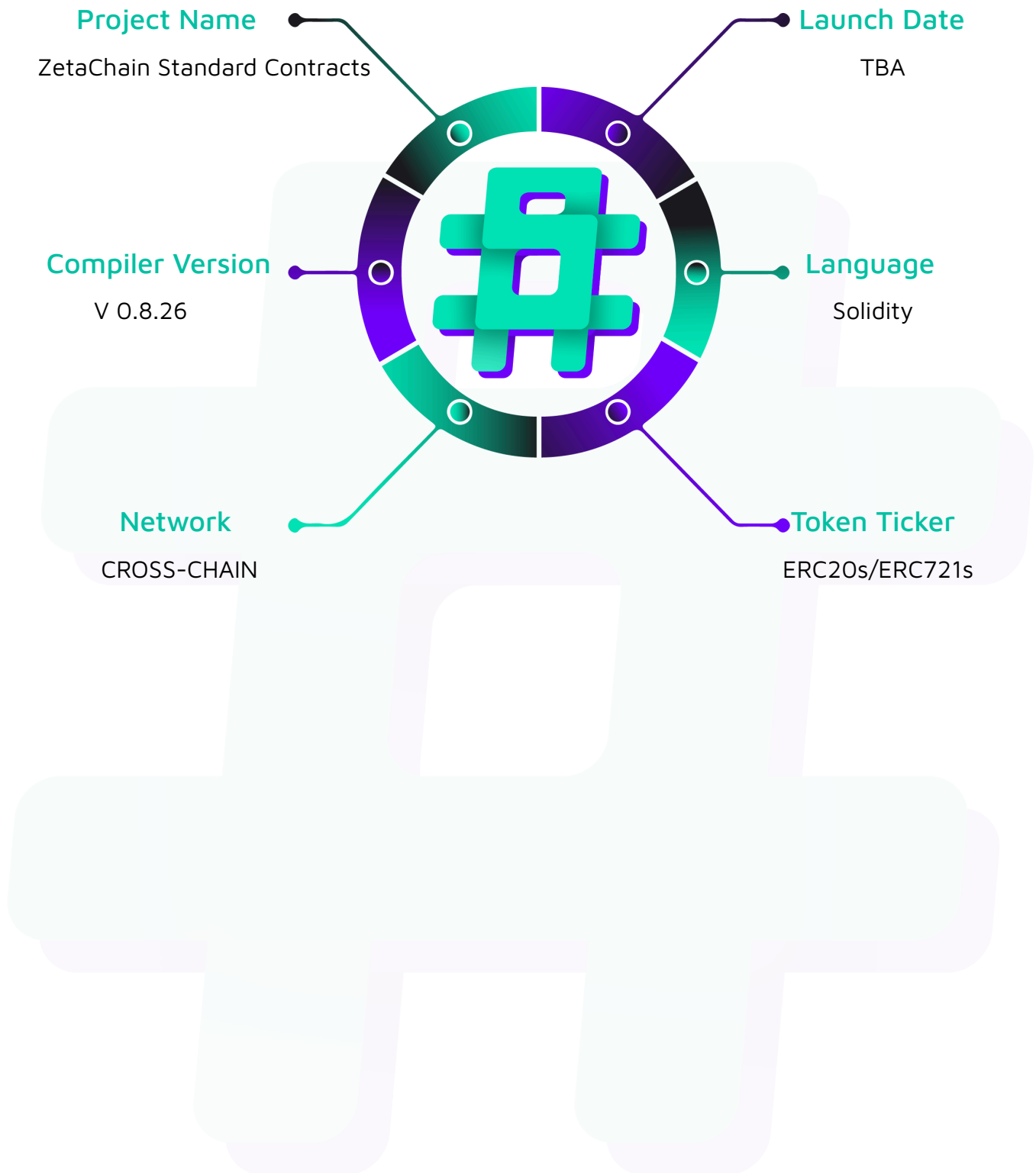
Project Type: Defi, Token, Cross-Chain

Compiler Version: 0.8.26

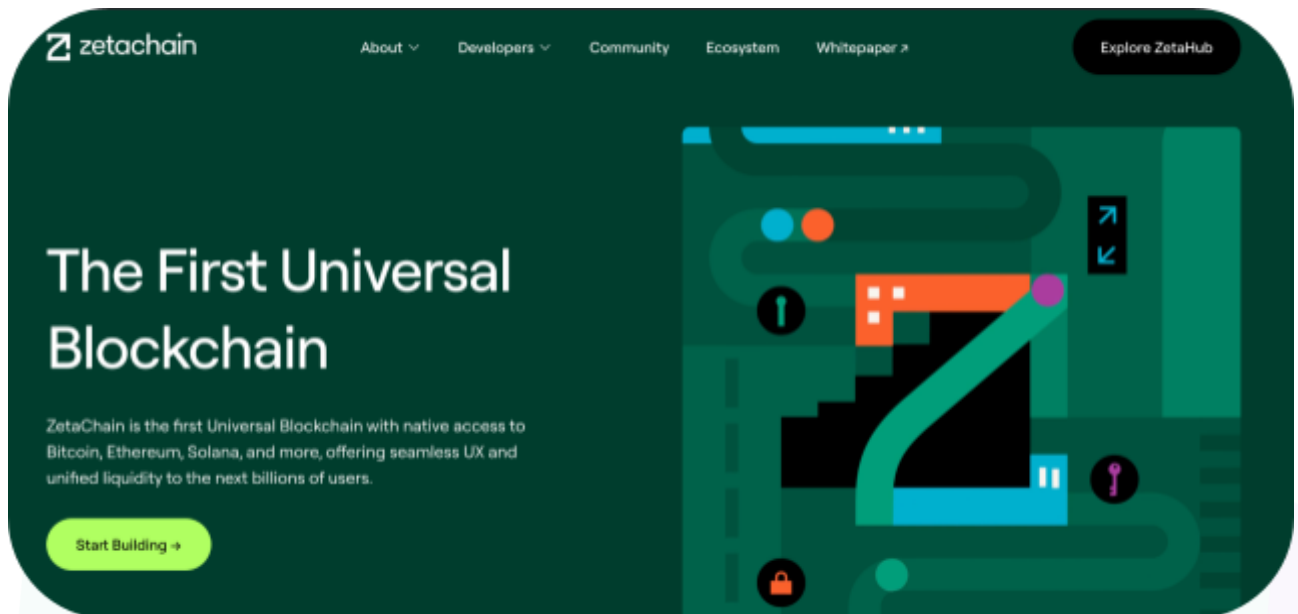
Website: www.zetachain.com

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the ZetaChain Standard Contracts, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Template Protocol Smart Contracts
Platform	Ethereum / Solidity
Audit Date	January, 2025
Contract 1	UniversalNFT.sol
Contract 1 MD5 Hash	23f0ce331ad2c7574c661ce4e5ece422
Contract 2	UniversalNFTCore.sol
Contract 2 MD5 Hash	1a89f37cbfffd0cfe48679e150e6ed85
Contract 3	EVMUniversalNFT.sol
Contract 3 MD5 Hash	358832ffa98888cc176eedbc1bffa71ab
Contract 4	ZetaChainUniversalNFT.sol
Contract 4 MD5 Hash	9a596829840c51c5ff6256984b0e905b
Contract 5	UniversalNFTEvents.sol
Contract 5 MD5 Hash	f9d6ebedd5487ee6cf7594a93d42cdda
Contract 6	UniversalNFT.sol
Contract 6 MD5 Hash	03ebb8e2d217069501e9634cf0af2d9c
Contract 7	UniversalNFTCore.sol
Contract 7 MD5 Hash	6f747adae5149104d08d6273f2b4c2cd
Contract 8	UniversalToken.sol
Contract 8 MD5 Hash	1c4280332702b255744f6875bb11ebc7

Contract 9	UniversalTokenCore.sol
Contract 9 MD5 Hash	71246956b7d02cd9a9cb5906a854de8f
Contract 10	EVMUniversalToken.sol
Contract 10 MD5 Hash	80645e1d8af4382c80bf3bf6f817b7bb
Contract 11	ZetaChainUniversalToken.sol
Contract 11 MD5 Hash	472649f5ddb1d1650afaccc80439f1e9
Contract 12	UniversalTokenEvents.sol
Contract 12 MD5 Hash	4a7a2658e7e28dd966e1ed19b1422fb8
Contract 13	UniversalToken.sol
Contract 13 MD5 Hash	1c4280332702b255744f6875bb11ebc7
Contract 14	UniversalTokenCore.sol
Contract 14 MD5 Hash	71246956b7d02cd9a9cb5906a854de8f
GitHub Commit Hash	9b99ad6b1b4df342dbcc5f9eef4f113519ab61d8

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some vulnerabilities that have since been addressed.

**Not Secure****Vulnerable****Secure****Hashlocked**

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Low severity vulnerability

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
UniversalTokenCore.sol - Enables ERC20 tokens to move across different blockchains - Handles token burning on source chain and minting on destination - Manage gas fees and token swaps through Uniswap - Core logic for cross-chain messaging via ZetaChain	Contract achieves this functionality.
UniversalNFTCore.sol - Enables ERC721 tokens to move across different blockchains - Preserves NFT metadata/URI across chains - Handles safe transfer checks and recipient validation - Manages cross-chain NFT ownership state	Contract achieves this functionality.
UniversalToken.sol - The actual deployable token contract - Implements pause/unpause for safety - Includes standard ERC20 functionality plus cross-chain features - Upgradeable using UUPS pattern	Contract achieves this functionality.
UniversalNFT.sol - Deployable NFT contract - Generates unique token IDs - Full ERC721 functionality plus cross-chain capabilities - Also upgradeable using UUPS	Contract achieves this functionality.

UniversalTokenEvents.sol - Defines events for token transfers, mints, and cross-chain operations - Enables tracking of token movement between chains - Includes universal contract setup events	Contract achieves this functionality.
UniversalNFTEvents.sol - Similar to TokenEvents but NFT-specific - Tracks individual NFT transfers across chains - Includes metadata/URI transfer events	Contract achieves this functionality.
Chain-specific variants - Adapted for chain-specific requirements - Handle Zeta-specific implementations - Include appropriate messaging protocols	Contracts achieve this functionality.
Example Contracts - Empty contracts used as examples	Contracts have no functionality

Code Quality

This audit scope involves the smart contracts of the ZetaChain Standard Contracts repo, as outlined in the Audit Scope section. All contracts, libraries, and interfaces follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the ZetaChain Standard Contracts smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] UniversalNFTCore and UniversalTokenCore - Missing zero address validation in setConnected functions

Description

The setConnected function in both UniversalTokenCore and UniversalNFTCore lacks zero address validation for its parameters. While these functions are owner-controlled, missing these basic checks could allow accidental misconfiguration of cross-chain token connections.

```
function setConnected(address zrc20, address contractAddress) external onlyOwner {  
    connected[zrc20] = contractAddress;  
    emit SetConnected(zrc20, contractAddress);  
}
```

Mistakenly setting an invalid zero address could break cross-chain transfers for tokens and NFTs and would lead to temporary system unavailability.

Recommendation

Add a zero address validation in the function in both contracts:

```
function setConnected(address zrc20, address contractAddress) external onlyOwner {  
    if (zrc20 == address(0)) revert ZeroAddress();  
    if (contractAddress == address(0)) revert ZeroAddress();  
    connected[zrc20] = contractAddress;  
    emit SetConnected(zrc20, contractAddress);  
}
```

Status

Resolved

QA**[Q-01] Inconsistent pragma version usage****Description**

The protocol shows inconsistent usage of pragma directives, with some contracts using a floating pragma `^0.8.26` while others use a locked pragma `0.8.26`. This inconsistency could lead to different compiler versions being used across the codebase, potentially introducing subtle incompatibilities or unexpected behaviors. After `0.8.26` there is concern about resurfacing bugs like the FullInliner non-expression-split argument evaluation order bug that was originally fixed in `0.8.21` but resurfaced in `0.8.26`.

source:

<https://soliditylang.org/blog/2024/05/21/solidity-0.8.26-release-announcement/>

Recommendation

Lock the pragma version across all contracts to a specific version to ensure consistency and predictable behavior.

Note

Hashlock states a concern about introducing inconsistent behavior due to compiler-level changes, potential incompatibilities between contracts, and making auditing and testing more complex due to version differences. This could lead to unexpected issues if/when future compiler versions are used.

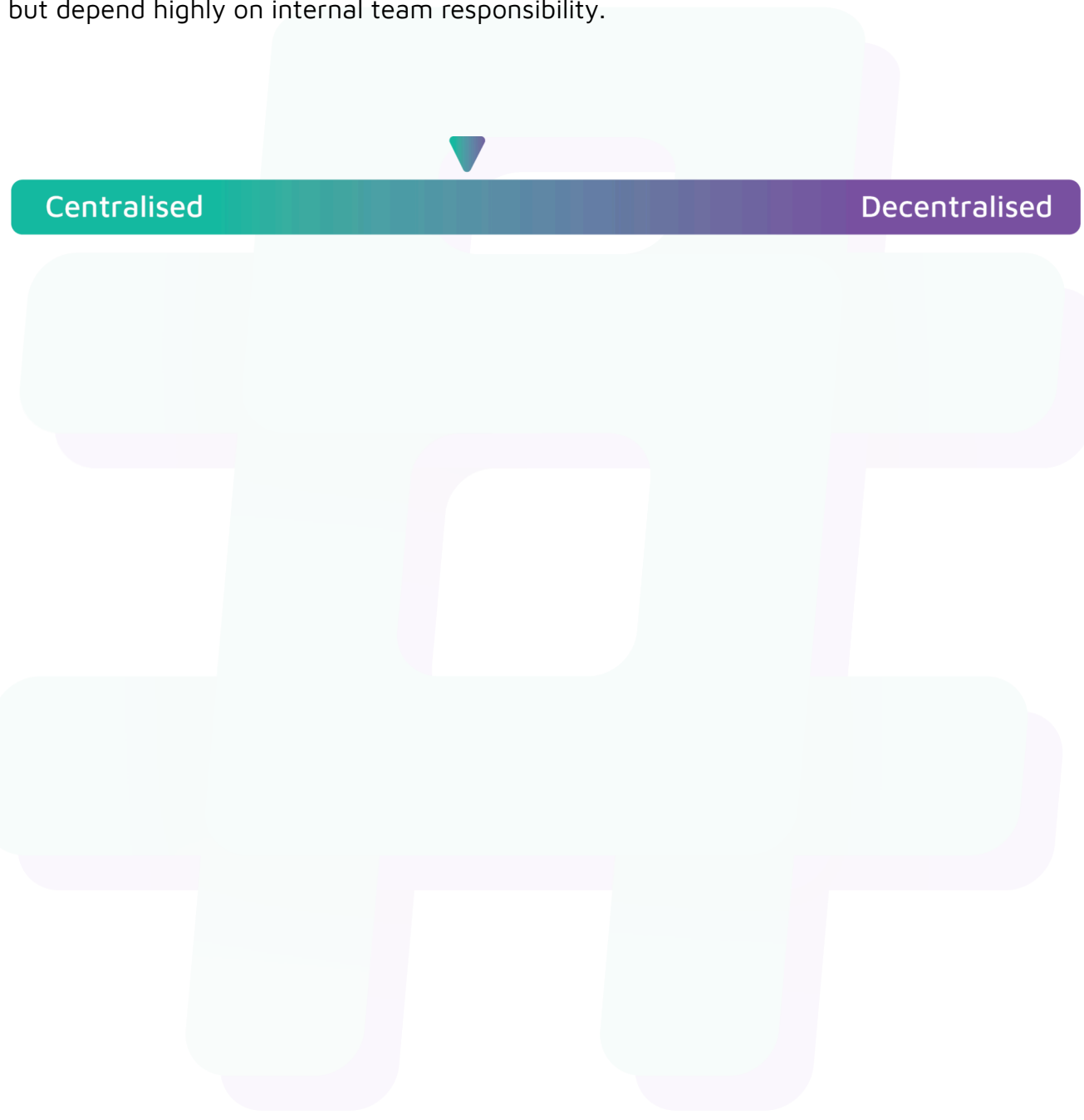
Status

Resolved

Centralisation

The ZetaChain Standard Contracts value security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the ZetaChain Standard Contracts seem to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.